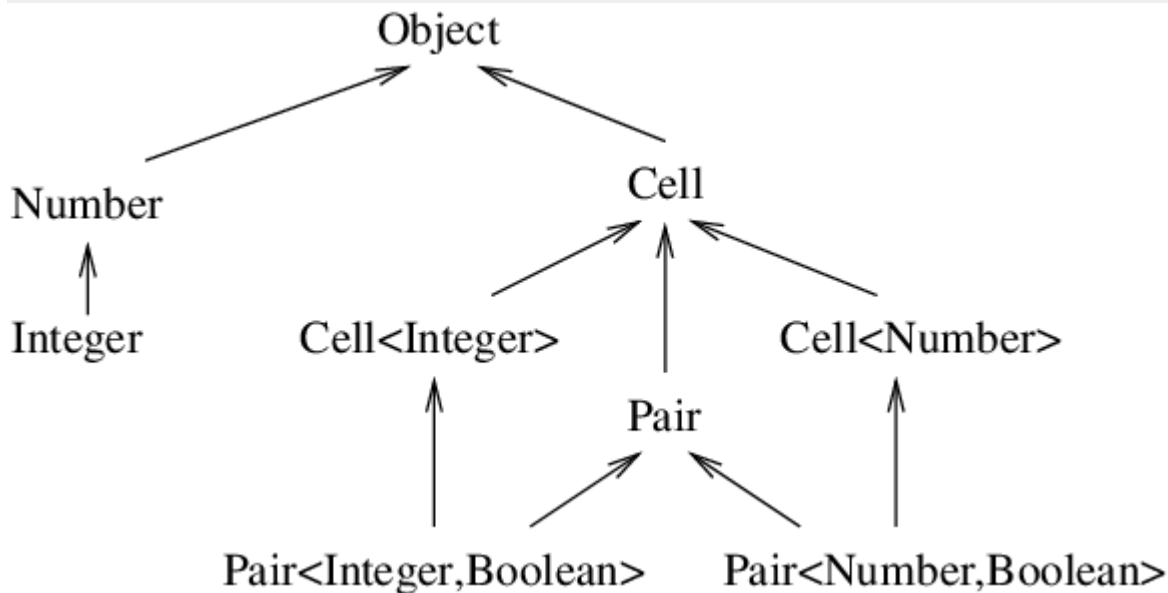


Tipos genéricos en Java



Un tipo genérico es una clase o interfaz genérica que se parametriza sobre tipos. La siguiente clase `Box` se modificará para demostrar el concepto.

La clase `Box`

Comenzamos examinando una clase `Box` no genérica que trabaja con objetos de cualquier tipo. Solo necesita proporcionar dos métodos: `set`, que agrega un objeto al *box*, y `get`, que lo recupera:

```
public class Box {  
    private Object object;  
  
    public void set(Object object) { this.object = object; }  
    public Object get() { return object; }  
}
```

Dado que sus métodos aceptan o devuelven un `Object`, se puede pasar lo que quiera, siempre que no sea uno de los tipos primitivos. No hay forma de verificar, en tiempo de compilación, cómo se usa la clase. Una parte del código puede colocar un número entero en la *box* y esperar obtener números enteros, mientras que otra parte del código puede pasar por error una cadena, lo que resulta en un error de tiempo de ejecución.

Versión genérica de la clase Box

Una clase genérica se define con el siguiente formato:

```
class name<T1, T2, ..., Tn> { /* ... */ }
```

La sección de parámetros de tipo, delimitada por corchetes angulares (<>), sigue el nombre de la clase. Especifica los parámetros de tipo (también llamados variables de tipo) T1, T2, ... y Tn.

Para actualizar la clase Box para usar genéricos, se crea una declaración de tipo genérico cambiando el código «public class Box» a «public class Box <T>». Esto introduce la variable de tipo, T, que se puede usar en cualquier lugar dentro de la clase.

Con este cambio, la clase Box se convierte en:

```
/**
 * Versión genérica de la clase Box
 * @param <T> el tipo del valor que se meterá dentro de la caja
 */
public class Box<T> {
    // T es el "tipo"
    private T t;

    public void set(T t) { this.t = t; }
    public T get() { return t; }
}
```

Como se puede ver, todas las apariciones de Object se reemplazan por T. Una variable de tipo puede ser cualquier tipo no primitivo que exista: cualquier tipo de clase, cualquier tipo de interfaz, cualquier tipo de matriz o incluso otra variable de tipo.

Esta misma técnica se puede aplicar para crear interfaces genéricas.

Convenciones de nomenclatura de parámetros de tipo

Por convención, los nombres de los parámetros de tipo son letras mayúsculas individuales. Esto contrasta fuertemente con las convenciones de nomenclatura habituales de variables, y con una buena razón: sin esta convención, sería difícil distinguir entre una variable de tipo y una clase ordinaria o nombre de interfaz.

Los nombres de parámetros de tipo más utilizados son:

- E – Elemento (usado ampliamente por el Java Collections Framework)
- K – Clave
- N – Número
- T – Tipo
- V – Valor
- S, U, V etc. – 2do, 3ro, 4to tipos

Veremos estos nombres utilizados en la API Java SE y el resto de esta lección.

Invocar e instanciar un tipo genérico

Para hacer referencia a la clase Box genérica desde su código, se debe realizar una invocación de tipo genérico, que reemplaza T con algún valor concreto, como Integer:

```
Box<Integer> integerBox;
```

Se puede pensar que una invocación de tipo genérico es similar a una invocación de método ordinario, pero en lugar de pasar un argumento a un método, está pasando un argumento de tipo Integer en este caso, a la clase Box.

Muchos desarrolladores usan los términos «parámetro de tipo» y «argumento de tipo» indistintamente, pero estos términos no son los mismos. Al codificar, se proporcionan argumentos de tipo para crear un tipo parametrizado. Por lo tanto, la T en Foo <T> es un parámetro de tipo y la Cadena en Foo <String> f es un argumento de tipo.

Al igual que cualquier otra declaración de variable, este código en realidad no crea un nuevo objeto Box. Simplemente declara que integerBox tendrá una referencia a una «Caja de enteros», que es cómo se lee Box <Integer>.

Una invocación de un tipo genérico se conoce generalmente como un tipo parametrizado.

Para crear una instancia de esta clase, use la nueva palabra clave, como de costumbre, pero coloque <Integer> entre el nombre de la clase y el paréntesis:

```
Box<Integer> integerBox = new Box<Integer>();
```

El diamante

En Java SE 7 y versiones posteriores, se pueden reemplazar los argumentos de tipo necesarios para invocar el constructor de una clase genérica con un conjunto vacío de argumentos de tipo (<>) siempre que el compilador pueda determinar o inferir los argumentos de tipo del contexto. Este par de corchetes angulares, <>, se llama informalmente el diamante. Por ejemplo, puede crear una instancia de `Box` `<Integer>` con la siguiente instrucción:

```
Box<Integer> integerBox = new Box<>();
```

Para más información sobre inferencia de tipos
ver: <https://docs.oracle.com/javase/tutorial/java/generics/genTypeInference.html>

Múltiples parámetros de tipo

Como se mencionó anteriormente, una clase genérica puede tener múltiples parámetros de tipo. Por ejemplo, la clase genérica `OrderedPair`, que implementa la interfaz genérica `Par`:

```
public interface Pair<K, V> {  
    public K getKey();  
    public V getValue();  
}  
  
public class OrderedPair<K, V> implements Pair<K, V> {  
  
    private K key;  
    private V value;  
  
    public OrderedPair(K key, V value) {  
        this.key = key;  
        this.value = value;  
    }  
  
    public K getKey() { return key; }  
    public V getValue() { return value; }  
}
```

Las siguientes declaraciones crean dos instancias de la clase `OrderedPair`:

```
Pair<String, Integer> p1 =  
    new OrderedPair<String, Integer>("Even", 8);  
Pair<String, String> p2 =  
    new OrderedPair<String, String>("hello", "world");
```

El código `new OrderedPair <String, Integer>`, instancia `K` como `String` y `V` como `Integer`. Por lo tanto, los tipos de parámetros del constructor de `OrderedPair` son `String` e `Integer`, respectivamente. Debido al autoboxing, es válido pasar un `String` y un `int` a la clase.

Como se menciona anteriormente, debido a que un compilador de Java puede inferir los tipos `K` y `V` de la declaración `OrderedPair <String, Integer>`, estas declaraciones pueden acortarse usando la notación de diamante:

```
OrderedPair <String, Integer> p1 =  
    new OrderedPair <> ("Even", 8);  
OrderedPair <String, String> p2 =  
    new OrderedPair <> ("hola", "mundo");
```

Para crear una interfaz genérica, seguiremos las mismas convenciones que para crear una clase genérica.

Tipos parametrizados

También se puede sustituir un parámetro de tipo (es decir, `K` o `V`) con un tipo parametrizado (es decir, `Lista <String>`). Por ejemplo, usando el ejemplo `OrderedPair <K, V>`:

```
OrderedPair<String, Box<Integer>> p =  
    new OrderedPair<>("primes", new Box<Integer>(...));
```

Este artículo está basado en el original en
Inglés: <https://docs.oracle.com/javase/tutorial/java/generics/types.html>